

ALGORITMI E STRUTTURE DATI

TECNICHE ALGORTIMICHE

AULA VIRTUALE DEL 25-05-2026

V 1.0

TECNICA DIVIDE ET IMPERA

Il metodo **divide et impera** è una tecnica algoritmica *top-down*, per progettare algoritmi asintoticamente efficienti. Con questo metodo si risolve un problema (istanza) in modo ricorsivo. Se il problema è sufficientemente piccolo (caso base), lo si risolve direttamente senza ricorsione, altrimenti (caso ricorsivo) si eseguono tre passaggi caratteristici.

Divide: il problema viene suddiviso in uno e più sottoproblemi, che sono istanze più piccole dello stesso problema.

Impera: i sottoproblemi vengono risolti in modo ricorsivo.

Combina: le soluzioni dei sottoproblemi vengono combinate per formare la soluzione del problema originale.

TECNICA DIVIDE ET IMPERA

Per analizzare gli algoritmi ricorsivi basati sul metodo **divide et impera**, abbiamo bisogno di alcuni strumenti matematici. Una ricorrenza è un'equazione che descrive una funzione in termini del suo valore su altri argomenti, in genere più piccoli.

Le ricorrenze vanno a braccetto con il metodo divide et impera perché ci danno un modo naturale per caratterizzare matematicamente i tempi di esecuzione degli algoritmi ricorsivi.

La forma generale di una ricorrenza è un'equazione o una disequazione che descrive una funzione definita sugli interi o sui reali utilizzando la funzione stessa. Essa contiene duo o più casi, a seconda dell'argomento. Se un caso comporta la chiamata ricorsiva della funzione di input (di solito più piccoli), si tratta di un **caso ricorsivo**. Se un caso non comporta una chiamata ricorsiva, è una **caso base**. Può non esistere nessuna, una o molte funzioni che soddisfano la ricorrenza. Una ricorrenza è **ben posta** se esiste almeno una funzione che la soddisfa altrimenti è **mal posta**.

TECNICA DIVIDE ET IMPERA

Abbiamo già introdotto questa tecnica algoritmica, dall'algoritmo di ricerca binaria e i classici algoritmi di ordinamento *mergeSort* e *quickSort*. Vogliamo trovare degli strumenti in grado di gestire le situazioni più comuni, ma al tempo stesso desideriamo avere anche strumenti generali che funzionino.

- Metodo di sostituzione
- Metodo dell'albero di ricorsione
- Metodo principale (Master Theorem)
- Metodo di Akra-Bazzi

TECNICA DIVIDE ET IMPERA

Metodo di sostituzione, facciamo un'ipotesi su quale potrebbe essere una stima asintotica e poi usiamo l'induzione matematica per dimostrare che la nostra ipotesi è corretta, e per determinare eventuali costanti. Questo metodo è forse quello formalmente più corretto per risolvere le ricorrenze, ma richiede anche di fare una buona ipotesi e di eseguire una dimostrazione per induzione.

Metodo dell'albero di ricorsione, rappresenta la ricorrenza con un albero i cui nodi rappresentano i costi ai vari livelli della ricorsione. Per risolvere la ricorrenza, si determinano i costi per livello e li si sommano, magari utilizzando le tecniche relative alla manipolazione di sommatorie. Anche se non si utilizza questo metodo per dimostrare formalmente una stima asintotica, può comunque essere utile per fare un'ipotesi sulla sua forma, per noi utilizzare questa ipotesi nel metodo di sostituzione.

Metodo principale, fornisce una stima asintotica per ricorrenze nella forma

$T(n) = aT(a/b) + f(n)$

TECNICA DIVIDE ET IMPERA

dove a>0 e b>1 sono costanti e f(n) è una funzione «forzante» assegnata (come nel caso di un'equazione differenziale non omogenea). Nello studio degli algoritmi, questo tipo di ricorrenze si presenta più frequentemente di ogni altro. Infatti ricorrenze di questo genere caratterizzano gli algoritmi del tipo divide et impera che creano a sottoproblemi, ciascuno dei quali di dimensione 1/b rispetto a quella del problema originale e in cui i passi divide e combina insieme richiedono un tempo f(n). Per utilizzare il metodo principale occorre memorizzare tre casi, ma una volta imparati, sarà facile determinare una stima asintotica per il tempo di esecuzione di molti algoritmi del tipo divide et impera.

Metodo di Akra-Bazzi, è un metodo generale per risolvere ricorrenze del tipo divide et impera. Sebbene richieda qualche nozione di analisi matematica, può essere utilizzato per affrontare ricorrenze più complicate di quelle risolte dal metodo principale.

TECNICA DIVIDE ET IMPERA

Quindi si tratta di dividere i dati di ingresso in due o più sottoinsiemi (**divide**), risolvere ricorsivamente il problema sui sottoinsiemi e poi ricombinare la soluzione dei sottoproblemi (**impera**) per ottenere la soluzione globale del problema originario.

Nel mergeSort, questi i passi della tecnica divide et impera:
Divide: dividi l'insieme da ordinare in due sottoinsiemi di cardinalità bilanciata, e ordinati ricorsivamente.
Impera: fondi due sottoinsiemi ordinati in un unico insieme ordinato.

Nel caso del quickSort, partizioniamo gli elementi in due sottoinsiemi che vengono ordinati ricorsivamente e che vengono poi ricombinati insieme. Differentemente dal mergeSort, tuttavia, in questo caso la parte più complessa è la fase di suddivisione (divide):
Divide: scelta di un elemento x della sequenza (il pivot), partiziona la sequenza in elementi <=x e in elementi >x , e ordina ricorsivamente le due sottosequenze.
Impera: Concatena le sottosequenze ordinate.

TECNICA DIVIDE ET IMPERA

Anche se mergeSort e quickSort sono entrambi di applicazioni della tecnica algoritmica divide et impera per lo stesso problema, differiscono profondamente nel modo in cui partizionano il problema e ricombinano le soluzioni.

Il **mergeSort** lavora con una partizione semplice, bilanciata sulla dimensione: la soluzione ricorsiva delle due parti produce due sottoinsiemi ordinati che sono «fusi» in un modo non banale nella fase di ricombinazione.

Il **quickSort** lavora invece con una partizione più complessa, basata sui valori di elementi, ma il passo di ricombinazione è semplice.

Questo per dire che l'applicazione di tale tecnica non può essere completamente automatica, ma spesso richiede uno **sforzo creativo** da parte del progettista di algoritmi nell'individuare tecniche **efficienti** di decomposizione del problema e di ricombinazione delle soluzioni.

TECNICA DIVIDE ET IMPERA: MERGE SORT

```
def merge_sort(lista):
    """
    Ordina una lista usando l'algoritmo Merge Sort.
    Merge Sort è un esempio di algoritmo divide et impera.
    """

    # Caso base:
    # se la lista ha 0 o 1 elementi, è già ordinata
    if len(lista) <= 1:
        return lista

    # DIVIDE:
    # troviamo il punto centrale della lista
    meta = len(lista) // 2

    # dividiamo la lista in due metà
    sinistra = lista[:meta]
    destra = lista[meta:]

    # IMPERA:
    # ordiniamo ricorsivamente le due metà
    sinistra_ordinata = merge_sort(sinistra)
    destra_ordinata = merge_sort(destra)
```

TECNICA DIVIDE ET IMPERA: MERGE SORT

```
    # COMBINA:
    # uniamo le due metà ordinate
    return merge(sinistra_ordinata, destra_ordinata)


def merge(sinistra, destra):
    """
    Unisce due liste già ordinate in un'unica lista ordinata.
    """

    risultato = []
    i = 0  # indice per la lista sinistra
    j = 0  # indice per la lista destra

    # Confrontiamo gli elementi delle due liste
    # e inseriamo ogni volta il più piccolo nel risultato
    while i < len(sinistra) and j < len(destra):
        if sinistra[i] <= destra[j]:
            risultato.append(sinistra[i])
            i += 1
        else:
            risultato.append(destra[j])
            j += 1
```

TECNICA DIVIDE ET IMPERA: MERGE SORT

```
    # Aggiungiamo eventuali elementi rimasti nella lista sinistra
    while i < len(sinistra):
        risultato.append(sinistra[i])
        i += 1

    # Aggiungiamo eventuali elementi rimasti nella lista destra
    while j < len(destra):
        risultato.append(destra[j])
        j += 1

    return risultato


# Esempio di utilizzo
numeri = [8, 3, 5, 1, 9, 2, 7, 4]

print("Lista originale:", numeri)

numeri_ordinati = merge_sort(numeri)

print("Lista ordinata:", numeri_ordinati)
```

TECNICA DIVIDE ET IMPERA: QUICK SORT

```
def quick_sort(lista):
    """
    Ordina una lista usando l'algoritmo Quick Sort.
    Quick Sort è un algoritmo basato sulla tecnica divide et impera.
    """

    # Caso base:
    # una lista vuota o con un solo elemento è già ordinata
    if len(lista) <= 1:
        return lista

    # DIVIDE:
    # scegliamo un elemento pivot.
    # Qui scegliamo l'elemento centrale della lista.
    pivot = lista[len(lista) // 2]

    # Creiamo tre liste:
    # - minori: elementi più piccoli del pivot
    # - uguali: elementi uguali al pivot
    # - maggiori: elementi più grandi del pivot
    minori = []
    uguali = []
    maggiori = []
```

TECNICA DIVIDE ET IMPERA: QUICK SORT

```
    for elemento in lista:
        if elemento < pivot:
            minori.append(elemento)
        elif elemento == pivot:
            uguali.append(elemento)
        else:
            maggiori.append(elemento)

    # IMPERA + COMBINA:
    # ordiniamo ricorsivamente minori e maggiori,
    # poi combiniamo il risultato:
    # lista ordinata = minori ordinati + pivot/uguali + maggiori ordinati
    return quick_sort(minori) + uguali + quick_sort(maggiori)


# Esempio di utilizzo
numeri = [8, 3, 5, 1, 9, 2, 7, 4]

print("Lista originale:", numeri)

numeri_ordinati = quick_sort(numeri)

print("Lista ordinata:", numeri_ordinati)
```

TECNICA DIVIDE ET IMPERA: RICERCA BINARIA

```
def ricerca_binaria(lista, valore):
    """
    Cerca un valore in una lista ordinata usando la ricerca binaria.
    Anche questo algoritmo usa la tecnica divide et impera.
    """

    inizio = 0
    fine = len(lista) - 1

    while inizio <= fine:
        # DIVIDE:
        # controlliamo l'elemento centrale
        centro = (inizio + fine) // 2

        # IMPERA:
        # se l'elemento centrale è quello cercato, abbiamo finito
        if lista[centro] == valore:
            return centro

        # Se il valore cercato è più piccolo,
        # continuiamo la ricerca nella metà sinistra
        elif valore < lista[centro]:
            fine = centro - 1
```

TECNICA DIVIDE ET IMPERA: RICERCA BINARIA

```
        # Se il valore cercato è più grande,
        # continuiamo la ricerca nella metà destra
        else:
            inizio = centro + 1

    # Se il ciclo termina, il valore non è presente
    return -1


# Esempio di utilizzo
numeri = [1, 2, 3, 4, 5, 7, 8, 9]

valore_da_cercare = 7
posizione = ricerca_binaria(numeri, valore_da_cercare)

if posizione != -1:
    print(f"Valore {valore_da_cercare} trovato in posizione {posizione}")
else:
    print(f"Valore {valore_da_cercare} non trovato")
```

PROGRAMMAZIONE DINAMICA

La programmazione dinamica, come il metodo divide et impera, risolve i problemi combinando le soluzioni dei sottoproblemi. In questo contesto, con il termine «programmazione» facciamo riferimento a un metodo di tabulazione, non alla scrittura di codice. Si tratta di una tecnica algoritmica di tipo *bottom-up*.

Come visto precedentemente gli algoritmi del tipo divide et impera suddividono un problema in sottoproblemi indipendenti, possono risolvere in modo ricorsivo i sottoproblemi e, poi, combinano le loro soluzioni per risolvere il problema originale.

La programmazione dinamica, invece, può essere applicata quando i sottoproblemi **non sono indipendenti**, ovvero quando i sottoproblemi hanno in comune dei **sottosottoproblemi**.

PROGRAMMAZIONE DINAMICA

In questo caso, un algoritmo di tipo divide et impera svolgerebbe molto più lavoro del necessario, risolvendo ripetutamente i **sottosottoproblemi** comuni a più **sottoproblemi**.

Un algoritmo che usa la programmazione dinamica risolve ciascun sottosottoproblema una sola volta e salva la sua soluzione in una tabella, evitando così di ricalcolare la soluzione ogni volta che si presenta il sottosottoproblema.

La programmazione dinamica, si applica, tipicamente, a problemi di ottimizzazione. Per questi problemi ci possono essere molte soluzioni possibili. Ogni soluzione ha un valore e vogliamo trovare una soluzione con il valore ottimale (massimo o minimo).

Diciamo, allora, che tale soluzione è una soluzione ottima del problema, non la soluzione ottima, perché ci possono essere più soluzioni che raggiungono il valore ottimo.

PROGRAMMAZIONE DINAMICA

Per sviluppare un algoritmo che faccia uso della programmazione dinamica, seguiamo una strategia formata dalle seguenti quattro fasi.

1. Caratterizzare la struttura di una soluzione ottima
2. Definire in modo ricorsivo il valore di una soluzione ottima
3. Calcolare il valore di una soluzione ottima, di solito con uno schema bottom-up
4. Costruire una soluzione ottima dalle informazioni che sono state calcolate.

Le fasi 1-3 formano la base per risolvere un problema applicando la programmazione dinamica. Se abbiamo bisogno solo del valore di una soluzione ottima, possiamo eliminare la fase 4. Quando eseguiamo la fase 4, spesso conviene memorizzare delle informazioni aggiuntive durante la fase 3 per semplificare la costruzione di una soluzione ottima.

PROGRAMMAZIONE DINAMICA: FIBONACCI

```
def fibonacci(n):
    """
    Calcola l'n-esimo numero di Fibonacci usando la programmazione dinamica.

    La sequenza di Fibonacci è:
    F(0) = 0
    F(1) = 1
    F(n) = F(n-1) + F(n-2)
    """

    # Caso base:
    # se n è 0 o 1, il risultato è direttamente n
    if n <= 1:
        return n

    # Creiamo una lista per salvare i risultati intermedi.
    # dp[i] conterrà il valore di Fibonacci(i)
    dp = [0] * (n + 1)
```

PROGRAMMAZIONE DINAMICA: FIBONACCI

```
    # Inizializziamo i casi base
    dp[0] = 0
    dp[1] = 1

    # Riempiamo la tabella dal basso verso l'alto.
    # Questo approccio si chiama "bottom-up".
    for i in range(2, n + 1):
        # Usiamo i risultati già calcolati invece di ricalcolarli
        dp[i] = dp[i - 1] + dp[i - 2]

    # Il risultato finale si trova in dp[n]
    return dp[n]


# Esempio di utilizzo
n = 10
risultato = fibonacci(n)

print(f"Il numero di Fibonacci in posizione {n} è {risultato}")
```

TECNICA GREEDY

La tecnica **greedy** viene tipicamente utilizzata per risolvere problemi di ottimizzazione, ovvero problemi per cui desideriamo trovare la migliore soluzione possibile. Alcuni problemi di ottimizzazione sono ad esempio trovare il percorso più breve per andare dalla città A alla città B, trovare l'ordine migliore in cui eseguire alcuni job in un computer, trovare l'ordine migliore di trasformare una stringa X in una stringa Y, oppure l'ordine migliore in cui eseguire il prodotto di n matrici. Nella situazione più generale, nella soluzione di un problema di ottimizzazione tramite la tecnica greedy avremo:

- Un insieme C di candidati possibili (ad esempio città, oppure job da eseguire);
- L'insieme dei candidati che sono già stati esaminati: tra questi distinguiamo i candidati definitivamente scartati e quelli selezionati per far parte di una soluzione ammissibile partiziale S;
- Una funzione di selezione (chiamata per esempio selezione) che indica quale dei candidati non ancora esaminati sia il più promettente.
- Una funzione ammissibile ammissibile che verifica se un insieme di candidati fornisce una soluzione (anche se non necessariamente ottima) al nostro problema. Ad esempio: un cammino non nessariamente ottimo dalla città A alla città B, oppure una trasformazione non necessariamente di costo minimo dalla stringa X alla stringa Y.

TECNICA GREEDY

- Una funzione ottimo che verifica se un insieme di candidati fornisce una soluzione ottima al nostro problema;
- Una funzione obiettivo che fornisce il valore di una soluzione (ad esempio, la lunghezza del percorso, il tempo necessario per eseguire tutti i job nell'ordine stabilito).

Per risolvere il problema di ottimizzazione, dovremo trovare un insieme di candidati che goda delle seguenti proprietà:

- L'insieme rappresenta una soluzione del problema;
- L'insieme ottimizza (i.e. minimizza o massimizza) il valore della funzione obiettivo.

Vediamo ora un esempio di algoritmo che sfrutta tecniche algoritmiche greedy, questo per esempio è il **problema del resto con il minor numero di monete (o distributore automatico di resto)**. L'idea greedy è: a ogni passo scegliamo la moneta più grande possibile che non supera l'importo rimanente (minor numero di monete).

TECNICA GREEDY: DISTRIBUTORE AUTOMATICO DI RESTO

Si prenda per esempio un distributore automatico di bibite, ogni bibita costa 37 centesimi di euro, noi introduciamo 1 euro per il pagamento, il resto sarà molto probabilmente composto da una moneta da 50 centesimi, una da 10 centesimi, una da 2 centesimi ed una da un 1 centesimo, per un totale di 63 centesimi di euro.
La ragione di questo comportamento potrebbe ad esempio essere un consumo uniforme delle tipologie di monete contenute nel serbatoio del distributore, onde evitare che esso rimanga privo di monete di un certo tipo. Supponiamo quindi che il distributore automatico abbia a disposizione, nel suo serbatoio, un certo numero di monete, ad esempio da 1, 2, 5, 10, 20 e 50 centesimi di Euro, e che debba erogare un resto R ad un cliente. Come fa il distributore a calcolare quali monete dare come resto?

Possiamo formulare questo problema nel paradigma greedy nel modo seguente:

TECNICA GREEDY: DISTRIBUTORE AUTOMATICO DI RESTO

- L'insieme di candidati possibili è dato da un insieme finito di monete, ad esempio da 1, 2, 5, 10, 20 e 50 centesimi di Euro, contenute nel serbatoio del distributore;
- La funzione ammissibile restituisce vero se il valore delle monete nell'insieme scelto non è superiore al resto che il distributore automatico deve restituire al cliente;
- La funzione ottimo restituisce vero se il valore delle monete nell'insieme scelto è esattamente uguale al resto che il distributore automatico deve restituire al cliente;
- La funzione seleziona sceglie la moneta di valore più grande rimasta tra quelle ancora da considerare;
- La funzione obiettivo restituisce il numero di monete presenti nella soluzione;
- La funzione valore restituisce il valore totale delle monete nell'insieme considerato.

Avremo quindi:

Ammissibile(S) <- (valore(S) <= R)
Ottimo(S) <- (valore(S) = R)
Seleziona(S) <- sceglie la moneta di valore più grande in C

TECNICA GREEDY

```
def resto_greedy(importo, monete):
    """
    Calcola il resto usando una tecnica greedy.

    Parametri:
    - importo: il valore da restituire
    - monete: lista dei tagli disponibili

    L'algoritmo sceglie ogni volta la moneta più grande possibile.
    """

    # Ordiniamo le monete in ordine decrescente
    # così proviamo prima i tagli più grandi
    monete.sort(reverse=True)

    # Lista che conterrà le monete usate per formare il resto
    risultato = []

    # Scorriamo tutti i tagli disponibili
    for moneta in monete:
```

PROGRAMMAZIONE DINAMICA: FIBONACCI

```
        # Finché la moneta corrente può essere usata,
        # la aggiungiamo al risultato
        while importo >= moneta:
            risultato.append(moneta)

            # Riduciamo l'importo rimanente
            importo -= moneta

    # Se importo è 0, siamo riusciti a formare il resto
    if importo == 0:
        return risultato
    else:
        return None


# Esempio di utilizzo
monete = [1, 2, 5, 10, 20, 50]
importo = 87

resto = resto_greedy(importo, monete)

if resto is not None:
    print("Monete usate:", resto)
    print("Numero di monete:", len(resto))
else:
    print("Non è possibile formare l'importo con le monete disponibili")
```

PROGRAMMAZIONE DINAMICA: FIBONACCI

Output:
Monete usate: [50, 20, 10, 5, 2]

Numero di monete: 5

In questo esempio l'algoritmo sceglie:

87 → prende 50, rimane 37
37 → prende 20, rimane 17
17 → prende 10, rimane 7
7 → prende 5, rimane 2
2 → prende 2, rimane 0

La tecnica greedy segue quindi una scelta locale ottima: prendere ogni volta la moneta più grande possibile.

Attenzione però: il greedy non garantisce sempre la soluzione ottima per tutti i sistemi di monete. Per esempio, con: monete = [1, 3, 4] con importo = 6

il greedy sceglierebbe:
4 + 1 + 1 = 3 monete
ma la soluzione ottima sarebbe:
3 + 3 = 2 monete

Quindi gli algoritmi greedy sono molto efficienti, ma funzionano correttamente solo quando il problema ha una struttura adatta.

ALGORITMI GENETICI

Gli algoritmi genetici sono tecniche di ottimizzazione ispirate all'evoluzione naturale. Si usano quando vogliamo trovare una buona soluzione a un problema, ma lo spazio delle soluzioni è molto grande e non conviene provarle tutte.
L'idea è simulare una popolazione di individui che evolvono nel tempo. Ogni individuo rappresenta una possibile soluzione.

A ogni generazione:

- si valuta quanto è buona ogni soluzione;
- si selezionano gli individui migliori;
- si combinano tra loro tramite crossover;
- si introducono piccole mutazioni casuali;
- si crea una nuova generazione.

ALGORITMI GENETICI

Concetti principali

Concetto	Significato
Individuo	Una possibile soluzione
Popolazione	Insieme di soluzioni candidate
Fitness	Valore che misura quanto una soluzione è buona
Selezione	Scelta degli individui migliori
Crossover	Combinazione di due soluzioni
Mutazione	Piccola modifica casuale
Generazione	Una fase dell'evoluzione

ALGORITMI GENETICI: FASI

Inizializzazione: creazione di una popolazione iniziale di cromosomi, spesso generata casualmente.

Selezione: processo di scelta dei cromosomi che verranno utilizzati per creare la nuova generazione, tecniche comuni includono la selezione per roulette e la selezione per torneo.

Crossover (Ricombinazione): operazione che combina parti di due cromosomi per creare uno o più nuovi cromosomi, simula la riproduzione sessuale.

Mutazione: modifica casuale di uno o più geni in un cromosoma per mantenere la diversità genetica all'interno della popolazione.

Valutazione: calcolo della fitness per ogni cromosoma nella popolazione.

Terminazione: criteri per fermare l'algoritmo, come un numero massimo di generazioni o una soglia di fitness.

ALGORITMI GENETICI: PROCESSO ALGORITMICO

1. Inizializzazione: generare una popolazione iniziale.
2. Valutazione: calcolare la fitness di ciascun cromosoma.
3. Selezione: selezionare i cromosomi per la riproduzione.
4. Crossover: eseguire il crossover tra i cromosomi selezionati.
5. Mutazione: applicare mutazioni ai cromosomi risultanti.
6. Sostituzione: creare una nuova popolazione sostituendo la vecchia con i nuovi cromosomi.
7. Ripetizione: ripetere il ciclo di valutazione, selezione, crossover, mutazione e sostituzione fino a soddisfare i criteri di terminazione.

ALGORITMI GENETICI: PRO

Pro

Vantaggio	Spiegazione
Adatti a problemi complessi	Funzionano bene quando lo spazio delle soluzioni è molto grande.
Non richiedono formule precise	Non serve conoscere una funzione matematica semplice o derivabile. Basta poter calcolare la fitness.
Buona esplorazione delle soluzioni	Grazie a crossover e mutazione esplorano molte aree diverse dello spazio di ricerca.
Utili con problemi non lineari	Possono essere applicati anche quando il problema ha molte variabili o vincoli complicati.
Flessibili	Si possono adattare a problemi di ottimizzazione, scheduling, percorsi, selezione, machine learning, ecc.
Parallelizzabili	La valutazione della fitness dei cromosomi può essere eseguita in parallelo.
Possono evitare minimi locali	La mutazione aiuta a uscire da soluzioni apparentemente buone ma non ottimali.

ALGORITMI GENETICI: CONTRO

Contro

Svantaggio	Spiegazione
Non garantiscono la soluzione ottima	Spesso trovano una buona soluzione, ma non necessariamente la migliore in assoluto.
Possono essere lenti	Se la popolazione è grande o la fitness è costosa da calcolare, richiedono molto tempo.
Dipendono molto dai parametri	Dimensione della popolazione, probabilità di mutazione e numero di generazioni influenzano molto il risultato.
Fitness difficile da progettare	Una funzione di fitness mal progettata può guidare l'algoritmo verso soluzioni scadenti.
Rischio di convergenza prematura	La popolazione può diventare troppo simile e bloccarsi su una soluzione non ottimale.
Risultati non sempre riproducibili	Essendo basati su casualità, esecuzioni diverse possono dare risultati diversi.
Richiedono molte prove	Spesso bisogna sperimentare varie configurazioni prima di ottenere buoni risultati.

ALGORITMI GENETICI

Gli algoritmi genetici sono molto utili quando il problema è difficile, grande o poco strutturato.

Sono però meno adatti quando:

- serve una soluzione esatta garantita;
- il problema è semplice e risolvibile con metodi classici;
- il tempo di calcolo è molto limitato;
- è difficile definire una buona funzione di fitness.

Gli algoritmi genetici sono potenti e flessibili, ma non garantiscono l’ottimo assoluto e richiedono una buona scelta dei parametri.

ALGORITMI GENETICI

```python
import random
import string


# Frase che vogliamo ottenere
OBIETTIVO = "ciao mondo"

# Caratteri che l'algoritmo può usare
CARATTERI = string.ascii_lowercase + " "

# Parametri dell'algoritmo genetico
DIMENSIONE_POPOLAZIONE = 100
PROBABILITA_MUTAZIONE = 0.01
NUMERO_GENERAZIONI = 1000


def crea_individuo():
    """
    Crea un individuo casuale.
    In questo caso, un individuo è una stringa lunga quanto la frase obiettivo.
    """
    return "".join(random.choice(CARATTERI) for _ in range(len(OBIETTIVO)))
```

ALGORITMI GENETICI

```python
def calcola_fitness(individuo):
    """
    Calcola quanto l'individuo è simile alla frase obiettivo.

    La fitness è il numero di caratteri corretti
    nella posizione corretta.
    """
    punteggio = 0

    for i in range(len(OBIETTIVO)):
        if individuo[i] == OBIETTIVO[i]:
            punteggio += 1

    return punteggio

def seleziona_genitore(popolazione):
    """
    Seleziona un genitore dalla popolazione.

    Qui usiamo una selezione semplice:
    scegliamo alcuni individui a caso e prendiamo il migliore tra loro.
    Questa tecnica si chiama selezione a torneo.
    """
    torneo = random.sample(popolazione, 5)

    migliore = max(torneo, key=calcola_fitness)

    return migliore
```

ALGORITMI GENETICI

```
def crossover(genitore1, genitore2):
    """
    Combina due genitori per creare un nuovo figlio.

    Prendiamo una parte dal primo genitore
    e una parte dal secondo genitore.
    """
    punto_taglio = random.randint(1, len(OBIETTIVO) - 1)
    figlio = genitore1[:punto_taglio] + genitore2[punto_taglio:]
    return figlio

def muta(individuo):
    """
    Applica una mutazione casuale all'individuo.

    Ogni carattere ha una piccola probabilità di essere sostituito
    con un altro carattere casuale.
    """
    nuovo_individuo = ""

    for carattere in individuo:
        if random.random() < PROBABILITA_MUTAZIONE:
            nuovo_individuo += random.choice(CARATTERI)
        else:
            nuovo_individuo += carattere
    return nuovo_individuo
```

ALGORITMI GENETICI

```
def algoritmo_genetico():
    """
    Esegue l'algoritmo genetico.
    """

    # Creiamo la popolazione iniziale in modo casuale
    popolazione = [crea_individuo() for _ in range(DIMENSIONE_POPOLAZIONE)]

    for generazione in range(NUMERO_GENERAZIONI):
        # Troviamo il miglior individuo della popolazione attuale
        migliore = max(popolazione, key=calcola_fitness)
        fitness_migliore = calcola_fitness(migliore)
        print(
            f"Generazione {generazione}: "
            f"{migliore} "
            f"(fitness = {fitness_migliore})"
        )

        # Se abbiamo raggiunto l'obiettivo, terminiamo
        if migliore == OBIETTIVO:
            print("Obiettivo raggiunto!")
            return migliore

        nuova_popolazione = []
```

ALGORITMI GENETICI

```
        # Creiamo una nuova popolazione
        while len(nuova_popolazione) < DIMENSIONE_POPOLAZIONE:
            # Selezione dei genitori
            genitore1 = seleziona_genitore(popolazione)
            genitore2 = seleziona_genitore(popolazione)

            # Crossover
            figlio = crossover(genitore1, genitore2)

            # Mutazione
            figlio = muta(figlio)

            # Aggiungiamo il figlio alla nuova popolazione
            nuova_popolazione.append(figlio)

        # La nuova popolazione sostituisce quella vecchia
        popolazione = nuova_popolazione

    return migliore


# Avvio dell'algoritmo
risultato = algoritmo_genetico()

print("Risultato finale:", risultato)
```

ALGORITMI E STRINGHE

Array di caratteri vengono anche definite stringhe. Approfondiremo gli algoritmi che risolvono problemi su stringhe, come ad esempio trovare la distanza tra due stringhe, trovare sottostringhe di una data stringa che godano di particolari proprietà, oppure ricercare le occorrenze di una data stringa all'interno del testo.

Si pensi al calcolo della distanza tra due stringhe è un problema che nasce nelle correzioni ortografiche automatiche (spell checking) dei documenti: in tal caso, i correttori ortografici (spell checker) non riconoscono soltanto un vocabolario errato in un documento, ma propongono o suggeriscono anche delle possibili correzioni o sostituzioni: la scelta di queste soluzioni si basa sulla ricerca di tutte le parole che sono simili, ovvero hanno una «distanza» piccola da quella scritta in modo errato.

ALGORITMI E STRINGHE

Gli algoritmi di string matching servono a cercare una stringa, detta pattern, all'interno di un testo più grande.

Ora supponiamo che siano assegnate due stringhe: un testo T e un pattern P, rispettivamente di lunghezza n > 0 e m > 0, con n >= m, abbiamo quindi la seguente…

Definizione (String Matching) Dati un pattern P e un testo T, il problema dello string matching consiste nel trovare se esiste un'occorrenza del pattern P nel testo T, ovvero se esiste un valore i, con 1 <= i <=(n – m +1), tale che P = T(i…i+m-1), ovvero P[j+1]=T[i+j] per ogni j,0 <= j <=(m-1). Se tale valore i esiste, allora diremo che il pattern P ha un match alla posizione i del testo T.

Esempio:
Testo:"abracadabra"
Pattern: "cad"
Il problema è capire se "cad" compare nel testo e, se sì, in quale posizione.

ALGORITMI E STRINGHE

```
def string_matching_bruteforce(testo, pattern):
    """
    Cerca tutte le occorrenze di pattern dentro testo
    usando l'algoritmo ingenuo, detto anche brute force.
    """

    posizioni = []

    # n è la lunghezza del testo
    n = len(testo)

    # m è la lunghezza del pattern
    m = len(pattern)

    # Scorriamo tutte le possibili posizioni iniziali
    # in cui il pattern potrebbe comparire nel testo
    for i in range(n - m + 1):

        # Supponiamo inizialmente che ci sia una corrispondenza
        trovato = True

        # Confrontiamo carattere per carattere
        for j in range(m):
            if testo[i + j] != pattern[j]:
                trovato = False
                break
```

ALGORITMI E STRINGHE

```
        # Se tutti i caratteri coincidono,
        # salviamo la posizione iniziale
        if trovato:
            posizioni.append(i)

    return posizioni


# Esempio di utilizzo
testo = "abracadabra"
pattern = "abra"

risultato = string_matching_bruteforce(testo, pattern)

print("Pattern trovato nelle posizioni:", risultato)
```

ALGORITMI E STRINGHE

Knuth-Morris-Pratt, KMP

L’algoritmo KMP evita di ricontrollare caratteri già confrontati. Usa una tabella chiamata spesso LPS, cioè Longest Prefix Suffix.

La tabella LPS indica, per ogni posizione del pattern, qual è la lunghezza del più lungo prefisso che è anche suffisso. Esempio:
pattern = "ababaca"

La tabella LPS aiuta a sapere di quanto spostare il pattern quando c’è un errore, senza ricominciare da zero.

ALGORITMI E STRINGHE

```
def kmp_search(testo, pattern):
    """
    Cerca tutte le occorrenze di pattern dentro testo
    usando l'algoritmo KMP.
    """

    posizioni = []

    n = len(testo)
    m = len(pattern)

    if m == 0:
        return posizioni

    # Calcoliamo la tabella LPS
    lps = calcola_lps(pattern)

    i = 0  # indice per il testo
    j = 0  # indice per il pattern

    while i < n:
        # Se i caratteri coincidono,
        # avanziamo sia nel testo sia nel pattern
        if testo[i] == pattern[j]:
            i += 1
            j += 1
```

ALGORITMI E STRINGHE

```
        # Se j arriva a m, abbiamo trovato un'occorrenza completa
        if j == m:
            posizioni.append(i - j)

            # Continuiamo la ricerca usando la tabella LPS
            j = lps[j - 1]

        # Se c'è un mismatch
        elif i < n and testo[i] != pattern[j]:
            if j != 0:
                # Usiamo LPS per evitare confronti inutili
                j = lps[j - 1]
            else:
                i += 1

    return posizioni


# Esempio di utilizzo
testo = "ababcabcababababd"
pattern = "ababd"

risultato = kmp_search(testo, pattern)

print("Pattern trovato nelle posizioni:", risultato)
```

ALGORITMI E STRINGHE

Rabin-Karp

L’algoritmo Rabin-Karp usa una funzione di hash. Invece di confrontare ogni volta il pattern carattere per carattere, confronta il valore hash del pattern con quello delle sottostringhe del testo. Se gli hash sono diversi, sicuramente le stringhe sono diverse. Se gli hash sono uguali, allora si fa un controllo carattere per carattere, perché possono esserci collisioni.

ALGORITMI E STRINGHE

```
def rabin_karp(testo, pattern):
    """
    Cerca tutte le occorrenze di pattern dentro testo
    usando l'algoritmo Rabin-Karp.
    """

    posizioni = []

    n = len(testo)
    m = len(pattern)

    if m == 0 or m > n:
        return posizioni

    # Base e modulo usati per il rolling hash
    base = 256
    modulo = 101

    # Calcoliamo l'hash del pattern
    hash_pattern = 0

    # Calcoliamo l'hash della prima finestra del testo
    hash_testo = 0

    # Valore usato per rimuovere il carattere più a sinistra
    h = 1
```

ALGORITMI E STRINGHE

```
for _ in range(m - 1):
        h = (h * base) % modulo

    for i in range(m):
        hash_pattern = (base * hash_pattern + ord(pattern[i])) % modulo
        hash_testo = (base * hash_testo + ord(testo[i])) % modulo

    # Scorriamo tutte le finestre del testo lunghe quanto il pattern
    for i in range(n - m + 1):

        # Se gli hash coincidono, controlliamo davvero i caratteri
        if hash_pattern == hash_testo:
            if testo[i:i + m] == pattern:
                posizioni.append(i)

        # Calcoliamo l'hash della finestra successiva
        if i < n - m:
            hash_testo = (
                base * (hash_testo - ord(testo[i]) * h)
                + ord(testo[i + m])
            ) % modulo

            # Evitiamo hash negativi
            if hash_testo < 0:
                hash_testo += modulo

    return posizioni
```

ALGORITMI E STRINGHE

```
# Esempio di utilizzo
testo = "abracadabra"
pattern = "abra"

risultato = rabin_karp(testo, pattern)

print("Pattern trovato nelle posizioni:", risultato)
```

ALGORITMI E STRINGHE

Python offre già strumenti integrati per cercare sottostringhe.

```
testo = "abracadabra"
pattern = "abra"

posizione = testo.find(pattern)

print(posizione)
```

ALGORITMI E STRINGHE

```
def trova_tutte_le_occorrenze(testo, pattern):
    """
    Trova tutte le occorrenze di pattern dentro testo
    usando il metodo find di Python.
    """

    posizioni = []

    posizione = testo.find(pattern)

    while posizione != -1:
        posizioni.append(posizione)

        # Continuiamo la ricerca dalla posizione
successiva
        posizione = testo.find(pattern, posizione + 1)

    return posizioni


testo = "aaaa"
pattern = "aa"

print(trova_tutte_le_occorrenze(testo, pattern))
```

ALGORITMI E STRINGHE

```
def string_matching_bruteforce(testo, pattern):
    """
    Cerca tutte le occorrenze di pattern dentro testo
    usando l'algoritmo ingenuo, detto anche brute
force.
    """

    posizioni = []

    # n è la lunghezza del testo
    n = len(testo)

    # m è la lunghezza del pattern
    m = len(pattern)

    # Scorriamo tutte le possibili posizioni iniziali
    # in cui il pattern potrebbe comparire nel testo
    for i in range(n - m + 1):

        # Supponiamo inizialmente che ci sia una
corrispondenza
        trovato = True
```

ALGORITMI E STRINGHE

```
            # Confrontiamo carattere per carattere
            for j in range(m):
                if testo[i + j] != pattern[j]:
                    trovato = False
                    break

            # Se tutti i caratteri coincidono,
            # salviamo la posizione iniziale
            if trovato:
                posizioni.append(i)

    return posizioni


# Esempio di utilizzo
testo = "abracadabra"
pattern = "abra"

risultato = string_matching_bruteforce(testo, pattern)

print("Pattern trovato nelle posizioni:", risultato)
```

ALGORITMI E STRINGHE

Confronto tra algoritmi

Algoritmo	Idea principale	Complessità media	Complessità peggiore
Brute force	Confronta in ogni posizione	$O(n \cdot m)$	$O(n \cdot m)$
KMP	Usa prefissi e suffissi	$O(n + m)$	$O(n + m)$
Rabin-Karp	Usa hash delle sottostringhe	$O(n + m)$	$O(n \cdot m)$
`find` di Python	Metodo integrato ottimizzato	Dipende dall'implementazione	Dipende dall'implementazione

ALGORITMI E STRINGHE

L’algoritmo Aho-Corasick è un algoritmo di string matching multiplo.

A differenza di KMP, che cerca un solo pattern alla volta, Aho-Corasick permette di cercare molti pattern contemporaneamente dentro un testo. Esempio:
Testo: "ushers"
Pattern: ["he", "she", "his", "hers"]

L’algoritmo trova in un’unica scansione:
"she" in posizione 1
"he" in posizione 2
"hers" in posizione 2

Aho-Corasick è come una versione “multi-pattern” di KMP. KMP usa la tabella LPS per un solo pattern, mentre Aho-Corasick usa un trie e i failure link per cercare molti pattern in una sola scansione del testo.

ALGORITMI E STRINGHE

Aho-Corasick costruisce una struttura dati chiamata trie, cioè un albero dei prefissi.Poi aggiunge dei collegamenti speciali chiamati failure link, simili all'idea della tabella LPS di KMP.

Differenza rispetto a KMP

KMP	Aho-Corasick
Cerca un solo pattern	Cerca più pattern insieme
Usa la tabella LPS	Usa failure link
Lavora su una singola stringa pattern	Lavora su un trie di pattern
Evita confronti inutili	Evita di tornare indietro nel testo

ALGORITMI E STRINGHE

```
from collections import deque


class Nodo:
    """
    Nodo del trie usato dall'algoritmo Aho-Corasick.
    """

    def __init__(self):
        # Dizionario: carattere -> nodo figlio
        self.figli = {}

        # Collegamento di fallimento.
        # Indica dove andare se non possiamo seguire il
carattere corrente.
        self.fallimento = None

        # Lista dei pattern che terminano in questo nodo
        self.output = []


class AhoCorasick:
    """
    Implementazione semplice dell'algoritmo Aho-Corasick.
    Permette di cercare più pattern contemporaneamente in un
testo.
    """
```

ALGORITMI E STRINGHE

```
def __init__(self, pattern):
    """
    Costruisce il trie e i failure link.
    """
    self.radice = Nodo()
    self.pattern = pattern

    self.costruisci_trie()
    self.costruisci_failure_link()

def costruisci_trie(self):
    """
    Inserisce tutti i pattern nel trie.
    """
    for parola in self.pattern:
        nodo = self.radice

        # Scorriamo i caratteri della parola
        for carattere in parola:
            # Se il carattere non esiste tra i figli,
            # creiamo un nuovo nodo
            if carattere not in nodo.figli:
                nodo.figli[carattere] = Nodo()

            # Ci spostiamo al nodo figlio
            nodo = nodo.figli[carattere]
```

ALGORITMI E STRINGHE

```
            # Alla fine della parola, salviamo il pattern
            nodo.output.append(parola)

    def costruisci_failure_link(self):
        """
        Costruisce i failure link usando una visita in
ampiezza, BFS.
        """

        coda = deque()

        # I figli diretti della radice hanno come failure
link la radice
        for figlio in self.radice.figli.values():
            figlio.fallimento = self.radice
            coda.append(figlio)

        # La radice fallisce su se stessa
        self.radice.fallimento = self.radice

        # BFS sui nodi del trie
        while coda:
            nodo_corrente = coda.popleft()
```

ALGORITMI E STRINGHE

```
            for carattere, figlio in
nodo_corrente.figli.items():
                coda.append(figlio)

                # Partiamo dal failure link del nodo
corrente
                fallback = nodo_corrente.fallimento

                # Cerchiamo il più lungo suffisso che possa
continuare
                # con il carattere corrente
                while fallback != self.radice and carattere
not in fallback.figli:
                    fallback = fallback.fallimento

                # Se troviamo un cammino con quel carattere,
                # il failure link del figlio punta lì
                if carattere in fallback.figli:
                    figlio.fallimento =
fallback.figli[carattere]
                else:
                    figlio.fallimento = self.radice
```

ALGORITMI E STRINGHE

```
                # Importante:
                # ereditiamo anche gli output del nodo di
fallimento.
                # Questo serve per trovare pattern che sono
suffissi
                # di altri pattern.
                figlio.output += figlio.fallimento.output

    def cerca(self, testo):
        """
        Cerca tutti i pattern nel testo.

        Restituisce una lista di tuple:
        (pattern_trovato, posizione_iniziale)
        """

        risultati = []
        nodo = self.radice

        for indice, carattere in enumerate(testo):

            # Se non possiamo seguire il carattere corrente,
            # usiamo i failure link finché troviamo una
transizione valida
            # oppure torniamo alla radice.
```

ALGORITMI E STRINGHE

```
            while nodo != self.radice and carattere not in
nodo.figli:
                nodo = nodo.fallimento

            # Se esiste una transizione con questo
carattere, la seguiamo
            if carattere in nodo.figli:
                nodo = nodo.figli[carattere]
            else:
                nodo = self.radice

            # Se il nodo corrente contiene output,
            # allora uno o più pattern sono stati trovati
            for parola in nodo.output:
                posizione_iniziale = indice - len(parola) +
1
                risultati.append((parola,
posizione_iniziale))

        return risultati


# Esempio di utilizzo
pattern = ["he", "she", "his", "hers"]
testo = "ushers"
```

ALGORITMI E STRINGHE

```
automa = AhoCorasick(pattern)

risultati = automa.cerca(testo)

for parola, posizione in risultati:
    print(f"Pattern '{parola}' trovato in posizione
{posizione}")
```

ALGORITMI E STRINGHE

Confronto con altri algoritmi

Algoritmo	Cosa cerca	Struttura usata	Complessità
Brute force	Un pattern	Nessuna	$O(n \cdot m)$
KMP	Un pattern	Tabella LPS	$O(n + m)$
Rabin-Karp	Uno o più pattern	Hash	Media $O(n + m)$, peggiore $O(n \cdot m)$
Aho-Corasick	Molti pattern	Trie + failure link	$O(n + m + z)$

ALGORITMI E STRINGHE

Lo string matching con automi a stati finiti è una tecnica per cercare un pattern dentro un testo costruendo prima un automa, cioè una macchina con stati e transizioni. In sintesi ogni stato rappresenta quanti caratteri del pattern sono stati riconosciuti fino a quel momento.

Dato:
Testo: "ababcabcababababd"
Pattern: "ababd"
vogliamo trovare tutte le posizioni in cui il pattern compare nel testo.

Supponiamo che il pattern sia:
ababd
La sua lunghezza è 5, quindi l'automa avrà gli stati:
0, 1, 2, 3, 4, 5

ALGORITMI E STRINGHE

Stato	Significato
0	Nessun carattere del pattern riconosciuto
1	Riconosciuto `"a"`
2	Riconosciuto `"ab"`
3	Riconosciuto `"aba"`
4	Riconosciuto `"abab"`
5	Riconosciuto `"ababd"`, quindi pattern trovato

ALGORITMI E STRINGHE

Lo stato finale è lo stato 5.
Quando raggiungiamo lo stato finale, significa che abbiamo trovato un'occorrenza del pattern.

Ogni volta che leggiamo un carattere del testo, l'automa cambia stato.

Esempio con pattern: ababd

Se siamo nello stato 3, significa che abbiamo già riconosciuto: aba

Se il prossimo carattere è "b", allora riconosciamo: abab
e passiamo allo stato 4.

Se invece il prossimo carattere non è quello atteso, non torniamo sempre a zero: l'automa cerca il più lungo suffisso già letto che sia anche prefisso del pattern.
Questa idea è simile a quella usata da KMP.

ALGORITMI E STRINGHE

```
def costruisci_automa(pattern, alfabeto):
    """
    Costruisce la tabella di transizione per lo string
matching
    con automa a stati finiti.

    pattern:
        stringa da cercare nel testo

    alfabeto:
        insieme dei caratteri che possono comparire nel
testo

    Ritorna:
        una tabella di transizione.
        transizioni[stato][carattere] = nuovo_stato
    """

    m = len(pattern)

    # L'automa ha stati da 0 a m.
    # Lo stato m è lo stato finale, cioè pattern
trovato.
    transizioni = {}
```

ALGORITMI E STRINGHE

```
    for stato in range(m + 1):
        transizioni[stato] = {}

        for carattere in alfabeto:
            # Proviamo ad aggiungere il carattere
corrente
            # alla parte di pattern già riconosciuta.
            stringa_corrente = pattern[:stato] +
carattere

            # Dobbiamo trovare il massimo k tale che:
            # pattern[:k] è suffisso di
stringa_corrente.
            nuovo_stato = 0

            for k in range(min(m,
len(stringa_corrente)), -1, -1):
                if
stringa_corrente.endswith(pattern[:k]):
                    nuovo_stato = k
                    break

            transizioni[stato][carattere] = nuovo_stato

    return transizioni
```

ALGORITMI E STRINGHE

```
def cerca_con_automa(testo, pattern):
    """
    Cerca tutte le occorrenze del pattern dentro il
testo
    usando un automa a stati finiti.
    """

    # L'alfabeto viene ricavato dal testo e dal
pattern.
    # In questo modo la tabella contiene solo i
caratteri necessari.
    alfabeto = set(testo + pattern)

    # Costruiamo la tabella di transizione
    transizioni = costruisci_automa(pattern, alfabeto)

    stato = 0
    posizioni = []
    m = len(pattern)

    # Leggiamo il testo un carattere alla volta
    for i, carattere in enumerate(testo):

        # Aggiorniamo lo stato usando la tabella di
transizione
        stato = transizioni[stato][carattere]
```

ALGORITMI E STRINGHE

```
            # Se raggiungiamo lo stato finale,
            # abbiamo trovato un'occorrenza del pattern
            if stato == m:
                posizione_iniziale = i - m + 1
                posizioni.append(posizione_iniziale)

    return posizioni


# Esempio di utilizzo
testo = "ababcababd"
pattern = "ababd"

risultato = cerca_con_automa(testo, pattern)

print("Pattern trovato nelle posizioni:", risultato)
```

ANALISI DEGLI ALGORITMI

Referenze

- Algoritmi e Strutture Dati – Demetrescu, Finocchi, Italiano
- Introduzione agli algoritmi e strutture dati – Cormen, Leiserson, Rivest, Stein

Risorse Python

https://runestone.academy/ns/books/published/pythonds/index.html?utm_source=chatgpt.com

https://github.com/0bprashanthc/algorithm-books/blob/master/Data%20Structures%20%26%20Algorithms%20in%20Python.pdf?utm_source=chatgpt.com

https://edu.anarcho-copy.org/Programming%20Languages/Python/Python%20Data%20Structures%20and%20Algorithms.pdf?utm_source=chatgpt.com

https://do1.dr-chuck.com/pythonlearn/IT_it/pythonlearn.pdf?utm_source=chatgpt.com

